Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 13/10/26**

A Mini Project Report on

Tablewise

A Multi-Restaurant Table Booking Platform with an AI Concierge, Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class _____

Submitted by

Name	Roll No.
Arya Bailur	2401008
Irisha Deshpande	2401022
Dev Chand	2401016

Under the guidance of

Prof. Dr. Sanober Shaikh_____

Department of Artificial Intelligence And Data Science College of Engineering

2026–2027

CERTIFICATE

This is to certify that the mini project entitled “Tablewise: A Multi-Restaurant Table Booking Platform with an AI Concierge, Delivered Through an Agile and DevOps Pipeline” is a bona fide work of

Arya Bailur (__2401001____), **Irisha Deshpande** (____2401022__) and

__Dev Chand____ (__2401016____), submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____ _____

Project Guide **Head of Department**

Date: 11/10/16_____ Place: _____

ABSTRACT

Booking a table in Mumbai still depends heavily on phone calls and walk-ins. Guests cannot see which restaurants have a free table for their party at a given time, and restaurants record allergies, occasions and no-shows inconsistently, often on paper. Tablewise addresses this with one web application in which a guest discovers restaurants, checks live availability, books a table with dietary details, and can do the same by chatting with an AI concierge.

The application was the vehicle for applying the seven experiments of the Agile Software Development & DevOps Lab. The work was planned in Jira as a Scrum project (key TAB) with 54 issues across two sprints, versioned in Git with a branch and pull request per issue, built and tested on every branch and pull request by a Jenkins multibranch pipeline (with GitHub Actions as a second CI), containerised with Docker, run as a 6-service Docker Compose stack, and monitored with Prometheus and Grafana.

The delivered system comprises a React 19 single-page application, a FastAPI backend with 14 REST endpoints, a LangGraph agent with retrieval-augmented generation over per-restaurant knowledge documents, and a Model Context Protocol server. The backend is covered by 42 automated tests at 71% line coverage, and every one of the 13 Jenkins builds recorded passed.

Keywords: Agile, Scrum, Jira, Git, GitHub Actions, Docker, Docker Compose, Jenkins, CI/CD, Prometheus, Grafana, FastAPI, React, LangGraph, RAG, MCP

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–5, 7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1.Background

Mumbai has thousands of restaurants spread across neighbourhoods such as Bandra, Colaba, Lower Parel, Juhu, Andheri and Powai, but reserving a table at most of them still means phoning the restaurant during service hours. A guest who wants “a rooftop Italian place in Bandra for four tonight” has to search on one site, call several restaurants and repeat the party size and time to each one. Restaurants, in turn, take bookings on paper or in a spreadsheet, so allergies, occasions and special requests are easily lost between the phone call and the table, and no-shows are not tracked at all.

The Agile Software Development & DevOps Lab introduced, through seven experiments, the practices used to plan, version, build, test, package, deploy and monitor software. This project applies those practices end to end to one application, so that each experiment is exercised on real code with real backlog items, branches, pull requests, builds and metrics rather than in isolation.

1.2.Problem Statement

To design, build and operate a web application that lets guests discover restaurants and book a table with their dietary needs, either through the user interface or by chatting with an AI concierge, and that gives hosts a live view of each evening’s bookings; and to deliver it through an Agile and DevOps pipeline covering backlog planning, version control, continuous integration, containerisation, orchestration, deployment and monitoring.

1.3.Objectives

1. Let a guest browse and filter restaurants by area, cuisine, price and tags, see the next free slots for their party, and book a table in a few steps.
2. Record allergies, dietary preference, occasion and special requests with every booking and show them to the host.
3. Provide an AI concierge that answers questions from each restaurant’s own documents (menu, policies, about) and can check availability, create, modify and cancel bookings through tool calls.
4. Expose the same booking tools to external AI clients through a Model Context Protocol (MCP) server.
5. Give hosts an admin timeline per restaurant with booking details and seated, completed and no-show statuses.
6. Plan the work as a Scrum backlog in Jira and link every branch, commit and pull request to an issue key.
7. Build, lint and test every branch and pull request automatically, and run the whole system with one Docker Compose command.

8. Monitor bookings, chat traffic, retrieval latency and LLM errors with Prometheus and Grafana.

1.4.Scope

The system covers 6 fictional restaurants in one city (Mumbai), each with its own opening hours, tables, booking-code prefix and knowledge documents. Payments, user accounts and authentication, reviews and delivery are out of scope: a booking is identified by its code, and the admin view is not protected (this is backlog item TAB-29). The AI concierge uses a hosted LLM through Groq.

1.5.Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in Tablewise	Section
1	Version control with Git	Branch per Jira issue, 6 pull requests, CODEOWNERS, protected main (1 approval), issue keys in 23 of 24 commits	5.1
2	Containerisation with Docker	Backend image (Python 3.11, health check, migrating entrypoint) and frontend image (multi-stage Node build served by Nginx)	5.2
3	Docker Compose	6-service stack with pinned images, health-gated start-up and named volumes	5.3
4	Jenkins CI pipeline	Dockerised Jenkins configured as code; multibranch pipeline over branches and pull requests; GitHub Actions as a second CI	5.4
5	Automated deployment	One-command deployment with automatic database migration and seeding; packaged build artifact per build	5.5
6	Agile project lifecycle	Jira Scrum project TAB, 54 issues, 2 sprints, issue keys in branches, commits, PRs and build comments	2
7	Monitoring	Prometheus metrics (HTTP and custom business metrics) and a provisioned Grafana dashboard	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1.User Roles

Role	Description	Main goals
Guest	A diner using the web application	Discover restaurants, book with dietary details, view or cancel a booking by code
Host / admin	Restaurant staff on the admin timeline	See the night's bookings, guest allergies and occasions; mark seated, completed, no-show or cancel
Concierge user	A guest chatting with the AI concierge	Ask about menus and policies, find restaurants, book or change a table in natural language
MCP client	An AI assistant such as Claude Desktop or Claude Code	Search restaurants and manage bookings through MCP tools

Table 2.1: User roles

2.2.Functional Requirements

- FR1 Discover: list restaurants with filters (area, cuisine, price level, tags, party size, date, time) and the next three free slots for each.
- FR2 Restaurant page: show about, menu and policies documents and a booking panel scoped to that restaurant.
- FR3 Availability: 30-minute slots between opening time and last seating; a slot is free when a table that fits the party has no overlapping confirmed or seated booking; the smallest fitting table is offered first.
- FR4 Booking: create, look up, modify and cancel bookings with a per-restaurant code (for example EMB-7K2P); store allergies, dietary preference, occasion, special requests and the booking source.
- FR5 Host workflow: per-restaurant timeline, a details drawer, and status changes between confirmed, seated, completed, no-show and cancelled.
- FR6 Concierge: classify intent, answer from retrieved documents with cited sources, search restaurants, and manage bookings through tools after confirming details with the guest.
- FR7 MCP: expose search and booking tools to external AI clients.

2.3.Non-Functional Requirements

- Portability: the complete system starts with a single `docker compose up -d --build`.
- Validation: malformed input (codes, dates, phones, emails, zones, statuses, allergy values) is rejected with HTTP 422 before it reaches the booking logic.
- Observability: request rates and latencies plus booking, chat, retrieval and LLM-error metrics are exposed on `/metrics` and charted in Grafana.

- Quality gates: every branch and pull request is linted, tested and built before review.
- Secrets: API keys are passed as environment variables and are never committed.

2.4.Scrum Setup

The project was run in Jira Software as a Scrum project with key **TAB** on board 69. Work was organised under five epics (Booking Engine; RAG & Knowledge Base; AI Concierge (LangGraph + MCP); Frontend & Google Maps Page; DevOps & Docs) and delivered in 2 sprints. Each story carries acceptance criteria; bugs carry steps to reproduce. Issue keys are used in branch names (for example `feature/TAB-50-admin-booking-details`), commit subjects, pull request titles and the comments Jenkins posts back to Jira after each build.

At the time of writing the project holds 54 issues: 5 epics, 46 stories and 3 bugs. Story points were recorded on 4 issues (6 points in total); the remaining stories were estimated informally, so velocity is reported in issues rather than points.

2.5.Product Backlog

Key	Type	Summary	Assignee	Status
TAB-6	Story	Data model for tables and bookings (SQLModel + SQLite)	Arya Bailur	Done
TAB-7	Story	Availability engine with 30-minute slots and smallest-fit table	Arya Bailur	Done
TAB-8	Story	Booking lifecycle with EV-XXXX codes	Arya Bailur	Done
TAB-9	Story	REST API for availability, bookings, tables and health	Arya Bailur	Done
TAB-10	Story	Fix naive datetime rejection in sqlmodel 0.0.48	Arya Bailur	Done
TAB-11	Story	Write the Ember & Vine knowledge base	Arya Bailur	Done
TAB-12	Story	Heading-aware chunking with source metadata	Arya Bailur	Done
TAB-13	Story	Idempotent ChromaDB ingest and retrieval with fastembed	Arya Bailur	Done
TAB-14	Story	Cite sources in concierge answers	Arya Bailur	Done
TAB-15	Story	LangGraph agent: router, retrieve, booking_agent, tools, respond	Arya Bailur	Done
TAB-16	Story	Stream graph events to the UI over SSE	Arya Bailur	Done
TAB-17	Story	Confirm-before-booking guardrail and relative dates	Arya Bailur	Done
TAB-18	Story	Switch LLM provider to Groq (openai/gpt-oss-120b)	Arya Bailur	Done
TAB-19	Story	MCP server exposing booking and search tools	Arya Bailur	Done
TAB-20	Story	Cinematic theme: gradient mesh, cursor orbs, grain and glass cards	Irisha Deshpande	Done
TAB-21	Story	Booking panel with date strip, party dial, slot pills and ticket	Irisha Deshpande	Done
TAB-22	Story	Concierge drawer with streaming chat, graph trace and tool cards	Irisha Deshpande	Done
TAB-23	Story	Admin timeline and dark/light theme	Irisha Deshpande	Done
TAB-24	Story	Interactive SVG floor plan	Irisha Deshpande	Done
TAB-25	Story	Show open times in the availability tool card	Irisha Deshpande	Done
TAB-26	Story	README, architecture diagrams and dev runner	Irisha Deshpande	Done
TAB-27	Story	Repo hygiene, MIT license and CI	Irisha Deshpande	Done
TAB-28	Story	Automated tests for booking service and agent graph	Arya Bailur	In Progress
TAB-29	Story	Protect the admin view	Arya Bailur	To Do

Key	Type	Summary	Assignee	Status
TAB-30	Story	Code-split the frontend bundle	Irisha Deshpande	Done
TAB-31	Story	Validate booking API input	Arya Bailur	In Progress
TAB-32	Story	Guardrail node for off-topic requests	Arya Bailur	In Progress
TAB-33	Story	Loading skeletons in the booking panel	Irisha Deshpande	In Progress
TAB-34	Story	Mobile polish for the booking panel	Irisha Deshpande	In Progress
TAB-35	Story	Jenkins stage for frontend lint	Irisha Deshpande	In Progress
TAB-36	Story	Booking confirmation notifications	Arya Bailur	To Do
TAB-37	Story	Deploy Tablewise (API + frontend)	Irisha Deshpande	Done
TAB-38	Story	Spike: Reserve with Google	Irisha Deshpande	Done
TAB-39	Bug	Admin view doesn't show new bookings made for other nights	Arya Bailur	In Progress
TAB-40	Story	Multi-restaurant data model	Arya Bailur	In Progress
TAB-41	Story	Per-restaurant RAG	Arya Bailur	In Progress
TAB-42	Story	Discover intent + search_restaurants tool	Arya Bailur	In Progress
TAB-43	Story	Discover and restaurant pages	Arya Bailur	In Progress
TAB-44	Story	Prometheus metrics	Arya Bailur	In Progress
TAB-45	Story	Grafana dashboard	Arya Bailur	In Progress
TAB-46	Story	Add GitHub Actions CI badge and CI section to README	Devv	To Do
TAB-47	Story	Add healthcheck stage to Jenkinsfile that curls /api/health after build	Devv	To Do
TAB-48	Story	Add make up / make down / make logs shortcuts for docker compose	Devv	To Do
TAB-49	Story	Add Grafana alert for p95 chat latency > 5s	Devv	To Do
TAB-50	Bug	Clicking a booking in the admin timeline doesn't show booking details	Arya Bailur	In Progress
TAB-51	Bug	Table names and booking codes unreadable in admin timeline (low contrast)	Arya Bailur	In Progress
TAB-52	Story	As a host, I can see guest allergies and dietary needs for each booking	Arya Bailur	In Progress
TAB-53	Story	As a guest, I can add allergies, dietary preferences and occasion when booking	Arya Bailur	In Progress

Key	Type	Summary	Assignee	Status
TAB-54	Story	As a host, I can mark a booking as seated, completed or no-show	Arya Bailur	In Progress

Table 2.2: Product backlog of project TAB (from Jira)

2.6.Sprint Plan

Sprint	Goal	Dates	State	Issues
MVP Build	Ship the Tablewise MVP: booking engine, RAG knowledge base, LangGraph + MCP concierge and cinematic UI.	2026-10-11 – 2026-10-25	Closed	20 (20 done)
Sprint 2 — Admin & Hardening	Fix issues found in Sprint 1 testing: booking details in admin, allergy tracking, timeline readability.	2026-10-11 – 2026-10-25	Active	26 (2 done)

Table 2.3: Sprint goals and scope

Sprint 1, **MVP Build**, delivered the single-restaurant booking engine, RAG knowledge base, LangGraph and MCP concierge, cinematic UI and the first CI setup; all 20 of its 20 issues were completed when the sprint was closed. The issues still open were carried into Sprint 2, **Admin & Hardening**, together with five issues raised during Sprint 1 testing (TAB-50 to TAB-54). Sprint 2 is active with 26 issues, 2 of them done; the rest are in review as open pull requests or not yet started.

2.7.Work Distribution

Table 2.4 shows ownership as recorded in Jira and authorship as recorded in Git. The two do not coincide: Jira assigns frontend and DevOps epics to Irisha Deshpande and the third member, but every commit in the repository was authored from Arya Bailur's account. The table reports both as they are rather than inferring who did which work.

Member	GitHub	Area	Jira issues assigned	Commits authored
Arya Bailur	aryabailur	Backend: booking engine, RAG, AI concierge, MCP	34 (14 done)	24
Irisha Deshpande	ahsiri14	Frontend and Google Maps page	15 (11 done)	0
(third member)	devv296	DevOps: Jenkins, Docker, GitHub Actions, CI/CD docs	5 (0 done)	0

Table 2.4: Distribution of work across the team

2.8.Definition of Done

- The change is on a branch named after its Jira issue and opened as a pull request with the issue key in the title.
- Backend lint (Ruff) and frontend lint (ESLint) pass with no errors.
- All backend tests pass, and new behaviour has tests.
- The Jenkins multibranch build for the branch and the pull request is green.

- The pull request is reviewed and approved by a teammate before merging into the protected main branch.
- The Jira issue carries the pull request link and is moved to Done only after the merge.

3. System Design

3.1.Architecture

Tablewise is a three-tier web application with an AI layer and an observability layer. The React single-page application is built into static files and served by Nginx, which also proxies `/api` (including the streamed chat responses) to the FastAPI backend. The backend owns the booking logic, the LangGraph agent and retrieval over a Chroma vector store, persists data in PostgreSQL and exposes Prometheus metrics. An MCP server reuses the same booking service and retrieval code directly.

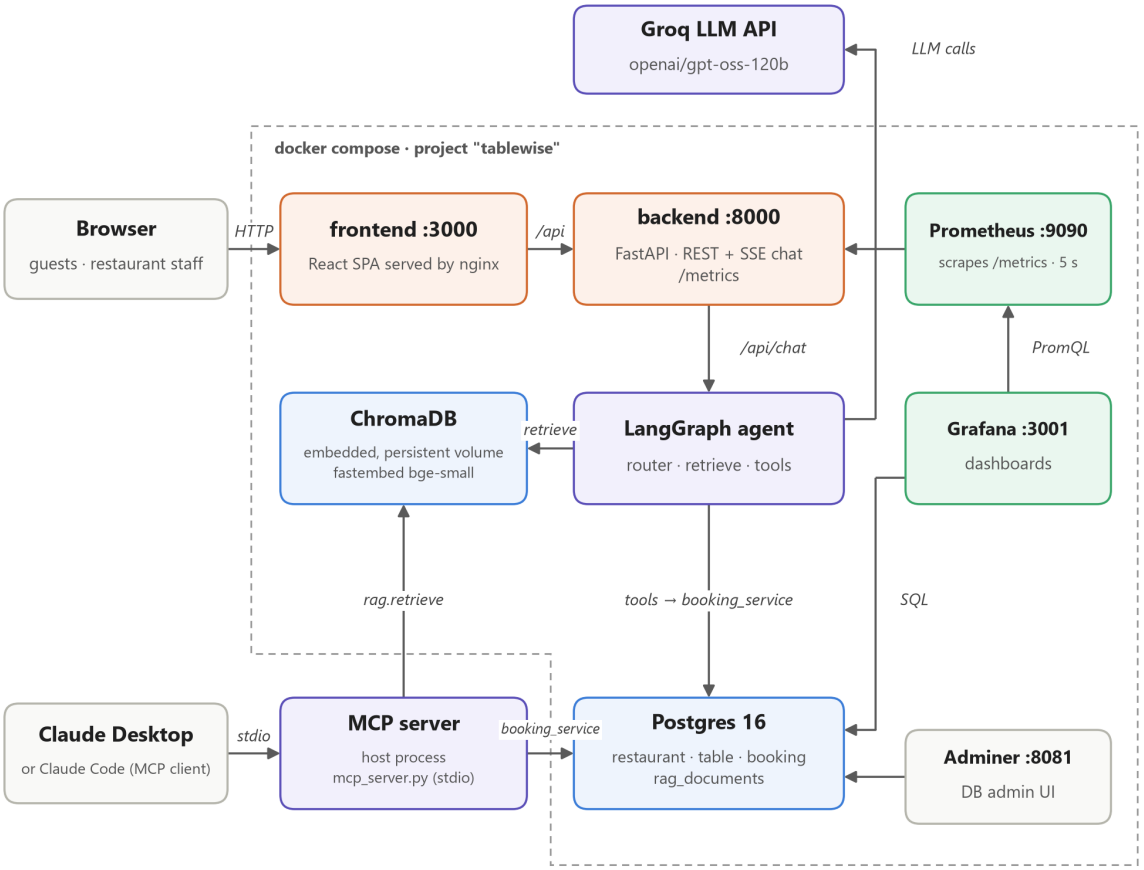


Figure 3.1: System architecture

3.2.Technology Stack

Layer	Technology	Purpose
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, Framer Motion, React Router	Discover, restaurant, booking, lookup and admin pages; concierge drawer
Backend	Python 3.11, FastAPI, SQLModel / SQLAlchemy, Pydantic	REST API, validation, booking service
AI agent	LangGraph, LangChain, Groq (openai/gpt-oss-120b)	Intent routing, tool calling, streamed answers
Retrieval	ChromaDB, fastembed (BAAI/bge-small-en-v1.5)	Per-restaurant document search
Database	PostgreSQL 16 (SQLite for local development)	Restaurants, tables, bookings, RAG documents
Agent interface	Model Context Protocol (Python SDK)	Booking and search tools for external AI clients
Containers	Docker, Docker Compose	Packaging and orchestration
CI	Jenkins (multibranch, JCasC), GitHub Actions	Lint, test, build on every branch and PR
Monitoring	Prometheus, Grafana, prometheus-fastapi-instrumentator	Metrics and dashboards
Planning	Jira Software (Scrum)	Backlog, sprints, issue tracking

Table 3.1: Technology stack

3.3.Database Design

Each restaurant owns its tables and bookings. A booking references one table, stores its start time and 90-minute default duration, a status, the channel it came from (web, chat, MCP, Google or phone) and the guest's allergies, dietary preference, occasion and special requests. The `rag_documents` table mirrors the chunks held in Chroma, keyed by restaurant slug, so retrieval content can also be inspected and charted from SQL.

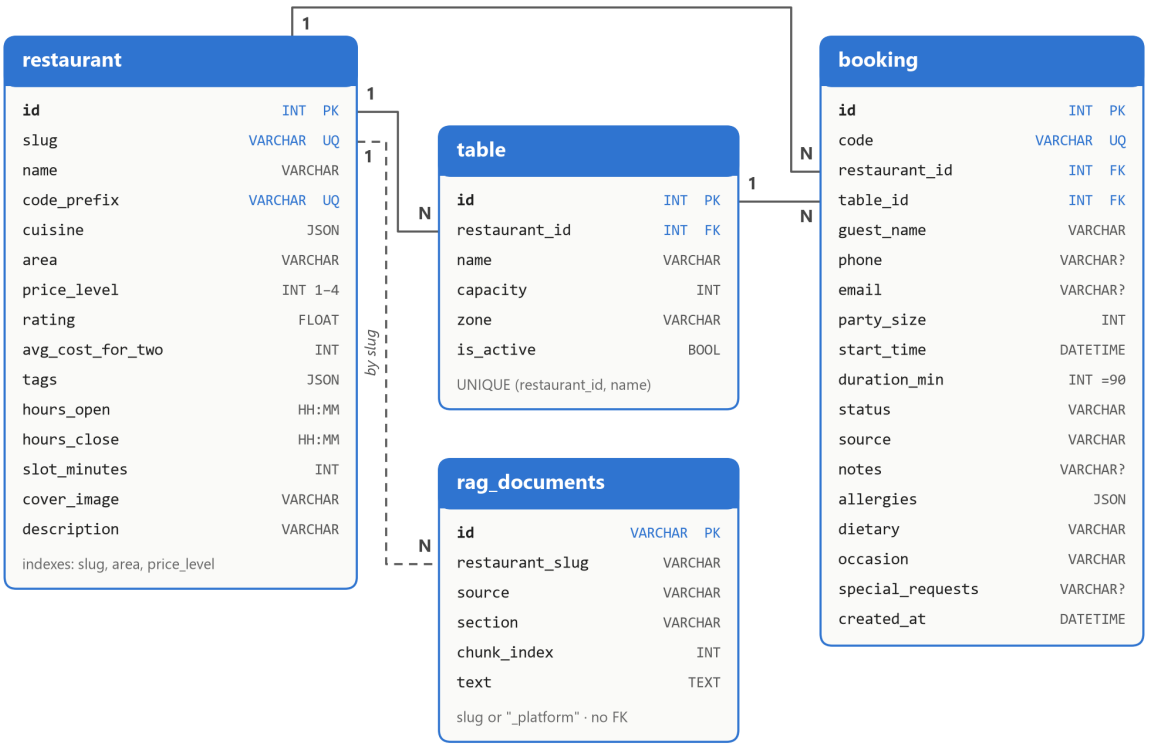


Figure 3.2: Entity-relationship diagram

3.4.Booking Lifecycle

A booking starts as confirmed. The host marks it seated when the party arrives, and completed when it leaves; a confirmed booking whose party never arrives becomes no-show. Only confirmed and seated bookings hold a table, so completing or marking a no-show frees the table for later slots. Cancelled is terminal: any further change returns HTTP 409, as does reopening a booking whose table has since been taken.

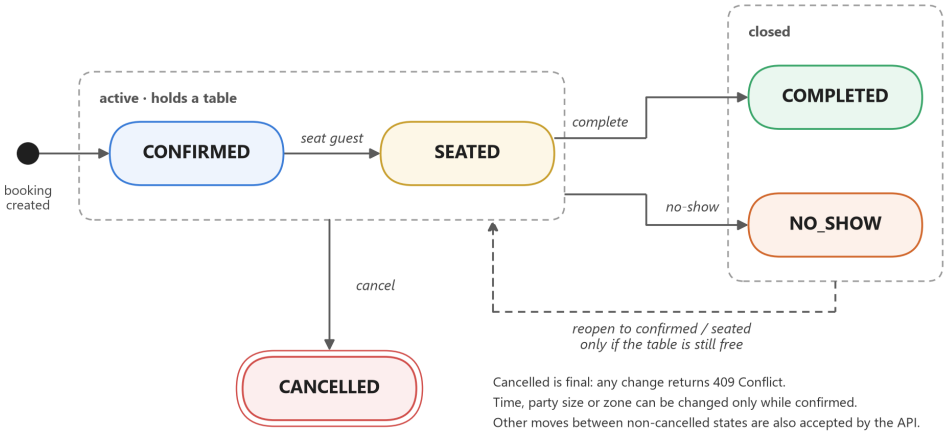


Figure 3.3: Booking state machine

3.5.API Design

Method	Path	Purpose
GET	`/api/health`	Service, LLM and data status
GET	`/api/restaurants`	Search with filters and next free slots
GET	`/api/restaurants/{slug}`	Restaurant details and documents
GET	`/api/restaurants/{slug}/tables`	Tables of a restaurant
GET	`/api/restaurants/{slug}/availability`	Slots for a date and party size
POST	`/api/restaurants/{slug}/bookings`	Create a booking
GET	`/api/tables`	Tables (legacy, default restaurant)
GET	`/api/availability`	Availability (legacy)
GET	`/api/bookings`	Bookings by date and restaurant (admin)
POST	`/api/bookings`	Create a booking (legacy)
GET	`/api/bookings/{code}`	Look up a booking
PATCH	`/api/bookings/{code}`	Modify details or status
DELETE	`/api/bookings/{code}`	Cancel a booking
POST	`/api/chat`	Concierge chat, streamed as Server-Sent Events

Table 3.2: REST API

3.6.Retrieval-Augmented Generation Design

Each restaurant has three Markdown documents (`about.md`, `menu.md`, `policies.md`) under `backend/knowledge/<slug>/`, and platform-wide questions are answered from `knowledge/_platform/faq.md`. Documents are split at headings into chunks of about 400 tokens with a 50-token overlap, embedded locally with fastembed and stored in one Chroma collection with the restaurant slug, source file and section as metadata. When the guest is on a restaurant page, retrieval filters on that slug plus the platform FAQ; discovery questions search all restaurants. The concierge cites the chunks it used, and the user interface shows them as source chips. The current index holds 130 chunks (_platform: 9, ember-vine: 22, kanji-house: 20, koli-tide: 21, saffron-terrace: 20, trattoria-alba: 21, tulsi-thali: 17).

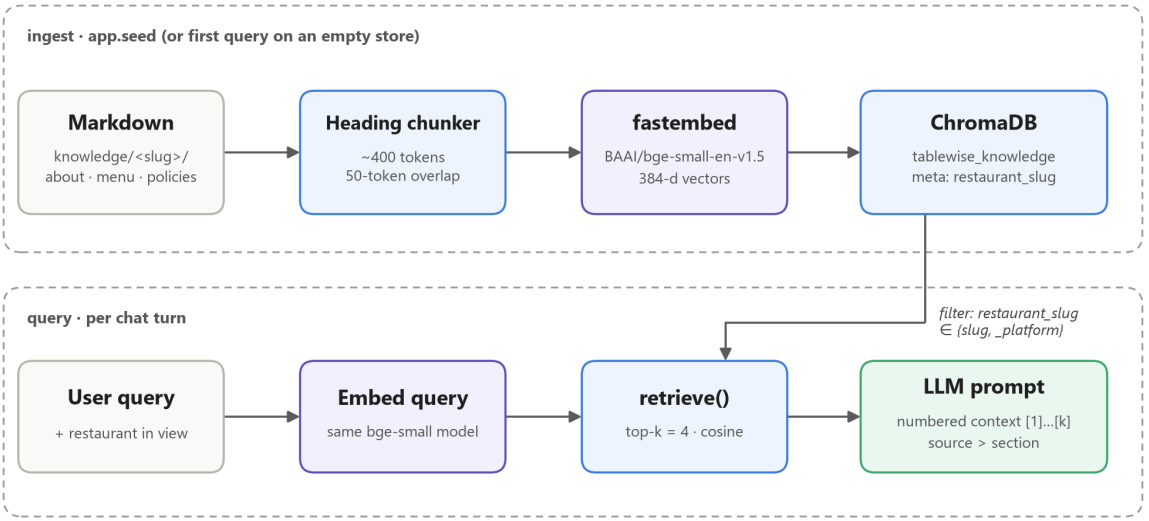


Figure 3.4: Ingestion and retrieval pipeline

3.7.Agent Design

The concierge is a LangGraph state graph. The **router** classifies each message as info, booking, discover, smalltalk or offtopic using structured LLM output, with a keyword fallback. Off-topic requests go to a **guardrail** node that replies without calling the LLM or any tool. Info, booking and discover messages go through **retrieve**; info questions are answered by **respond**, while booking and discovery go to **booking_agent**, an LLM bound to the tools listed below, which loops through a **tools** node until it has what it needs. The graph state carries the conversation, intent, retrieved documents, tool results, a booking draft and the current restaurant, and is checkpointed per chat thread. The agent must recap a booking and receive an explicit confirmation before calling `create_booking`, asks about allergies, and names menu dishes the guest should avoid.

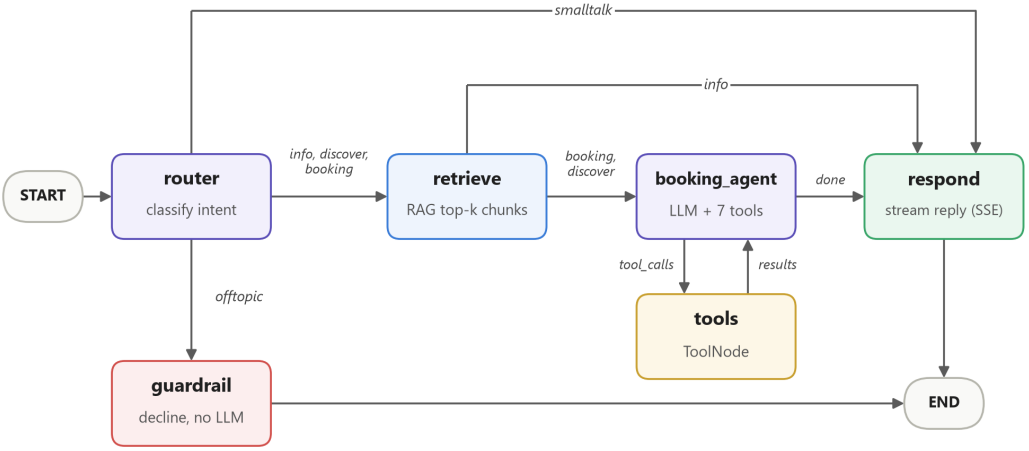


Figure 3.5: LangGraph concierge graph

Tool	Purpose
`search_restaurants`	Filter restaurants and return next free slots
`check_availability`	Free times and suggested table for a date and party
`create_booking`	Create a confirmed booking after the guest confirms
`modify_booking`	Change time, party size, zone, details or status
`check_menu_allergens`	Dishes on the restaurant menu that contain an allergen
`cancel_booking`	Cancel a booking by code
`lookup_booking`	Fetch a booking by code

Table 3.3: Agent tools

3.8.MCP Server

The MCP server (`backend/mcp_server.py`) runs over standard input and output, so Claude Desktop or Claude Code can start it as a local process. It imports the booking service and retrieval modules directly instead of calling the REST API, and records bookings it creates with source `mcp`.

Tool	Purpose
`search_restaurants`	Search restaurants with filters and open slots
`search_restaurant_info`	Retrieve knowledge-base passages, optionally for one restaurant
`check_availability`	Availability for a restaurant, date and party
`create_booking`	Create a booking with dietary details
`get_booking`	Look up a booking by code
`modify_booking`	Modify a booking or its status
`cancel_booking`	Cancel a booking

Table 3.4: MCP server tools

4. Application Implementation

This chapter walks through the application as a guest and as a host would use it. All screenshots are taken from the running Docker Compose stack with seeded demonstration data.

4.1.Discover Page

The home page lists the six restaurants as cards showing the cuisine, area, rating, cost for two and the next three free slots for the chosen date and party size. Filter chips narrow the list by area, cuisine, price level and tags such as rooftop or pure-veg; the filters live in the URL, so the back button and shared links keep them. Selecting a slot opens the restaurant with that slot preselected.

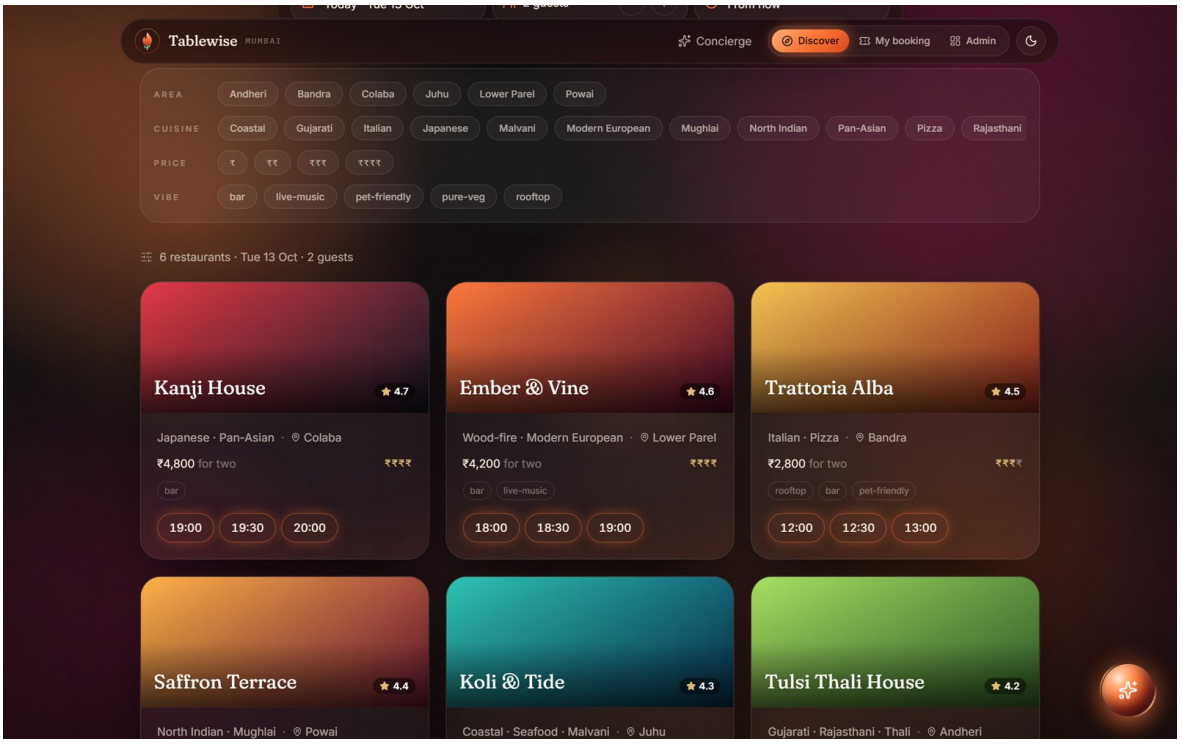


Figure 4.1: Discover page

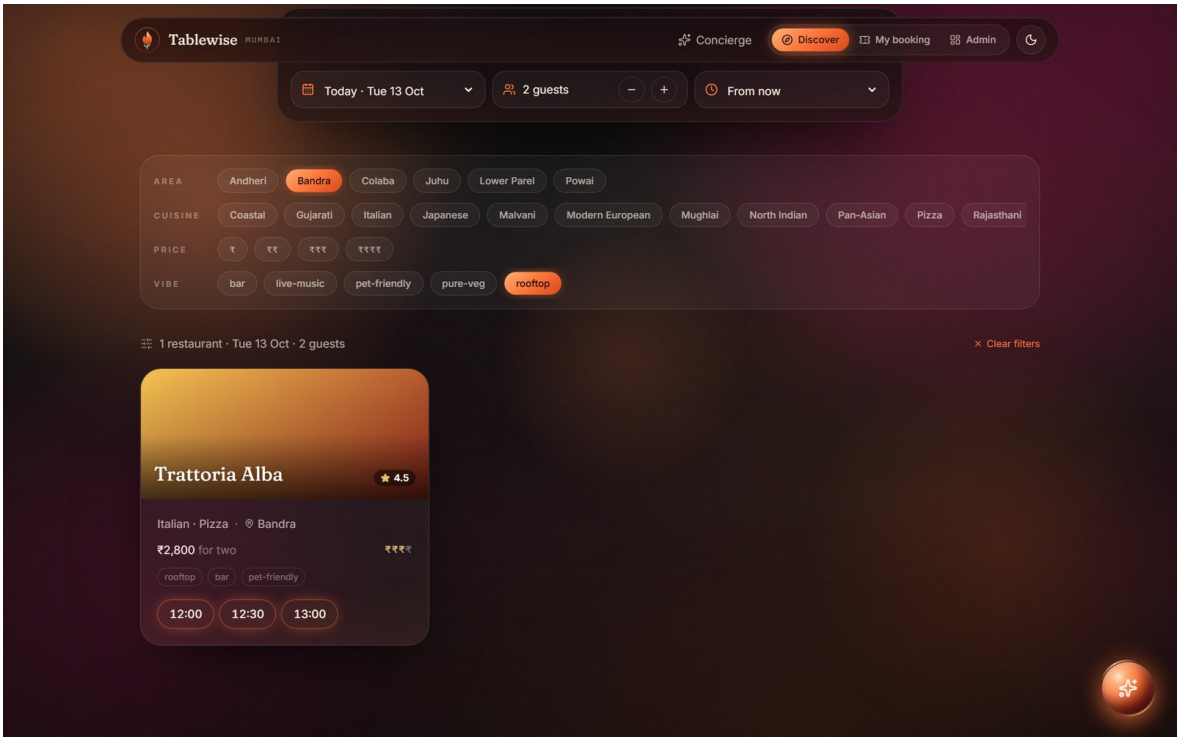


Figure 4.2: Discover page with a filter applied

4.2.Restaurant Page

Each restaurant page shows a cover, the restaurant's details and tabs for About, Menu and Policies, rendered from the same Markdown documents the concierge retrieves from. The page also injects schema.org Restaurant data (JSON-LD) with a reservation action.

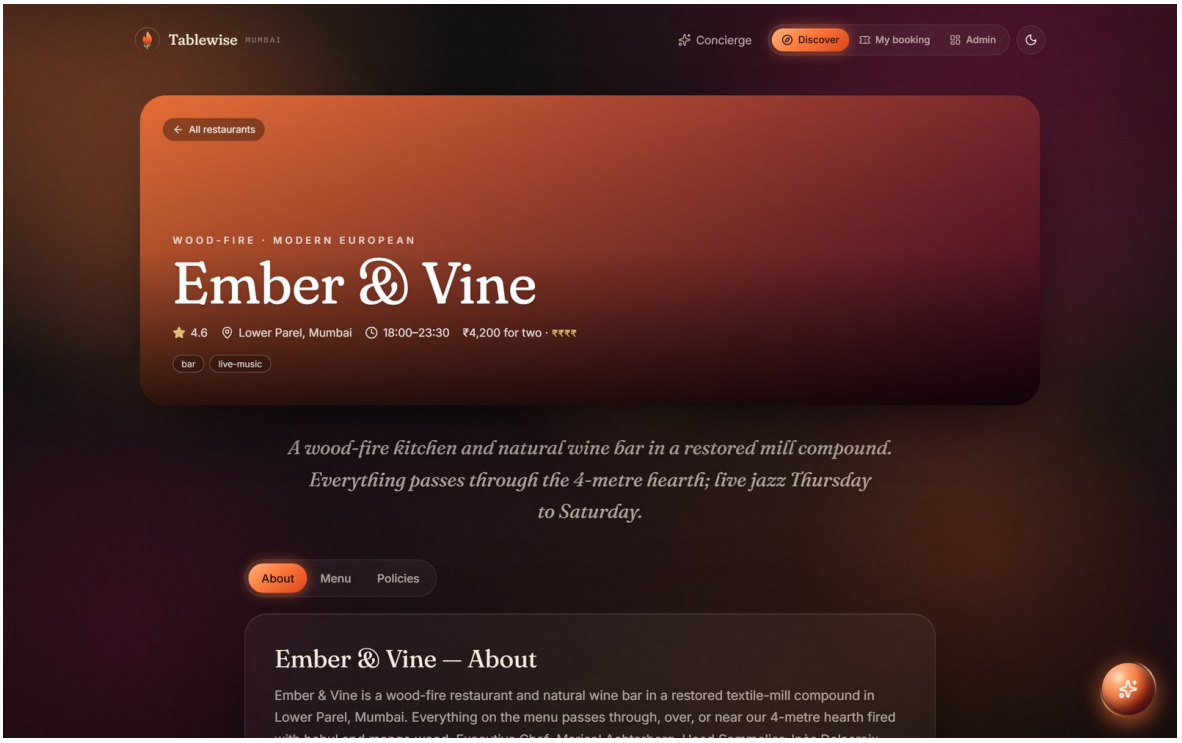


Figure 4.3: Restaurant page

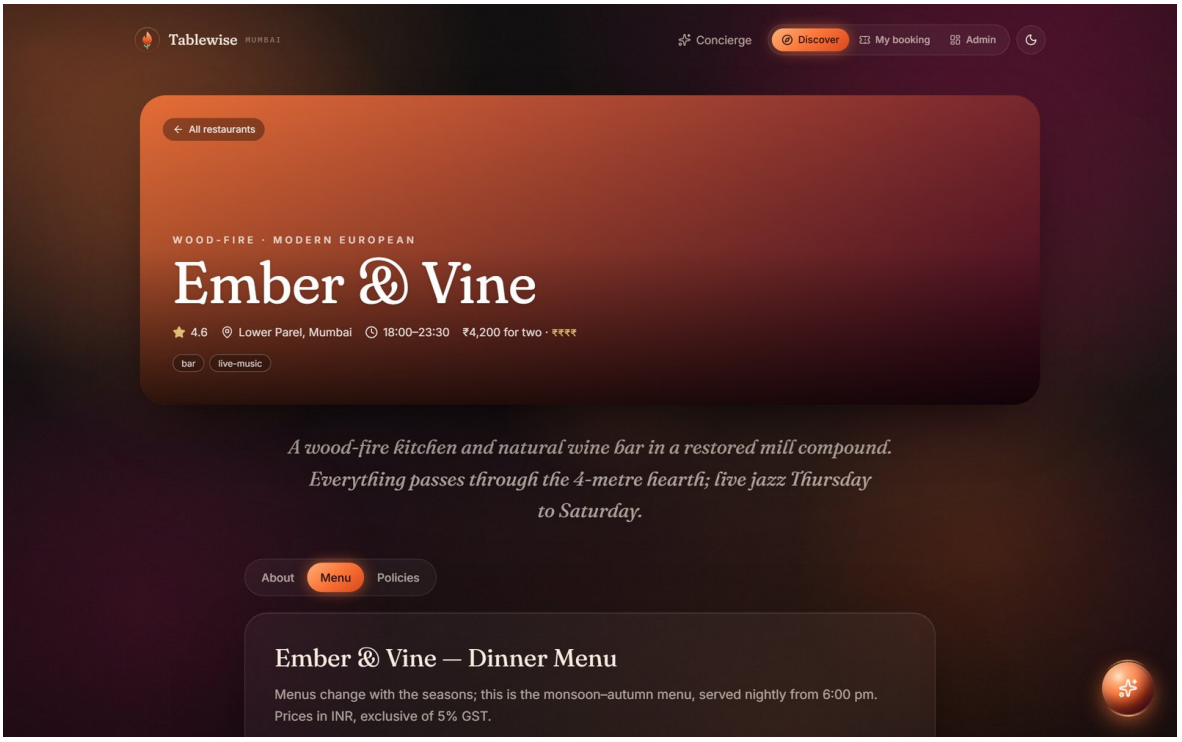


Figure 4.4: Menu tab rendered from the knowledge documents

4.3.Booking Flow with Dietary Details

The booking panel steps through date, party size, time and table, then an optional preferences step with allergy chips (nuts, peanuts, gluten, dairy, shellfish, fish, eggs, soy, sesame), a dietary preference, an occasion and special requests, and finally the guest's name and contact. The backend re-validates every value and returns HTTP 422 for anything outside the allowed sets.

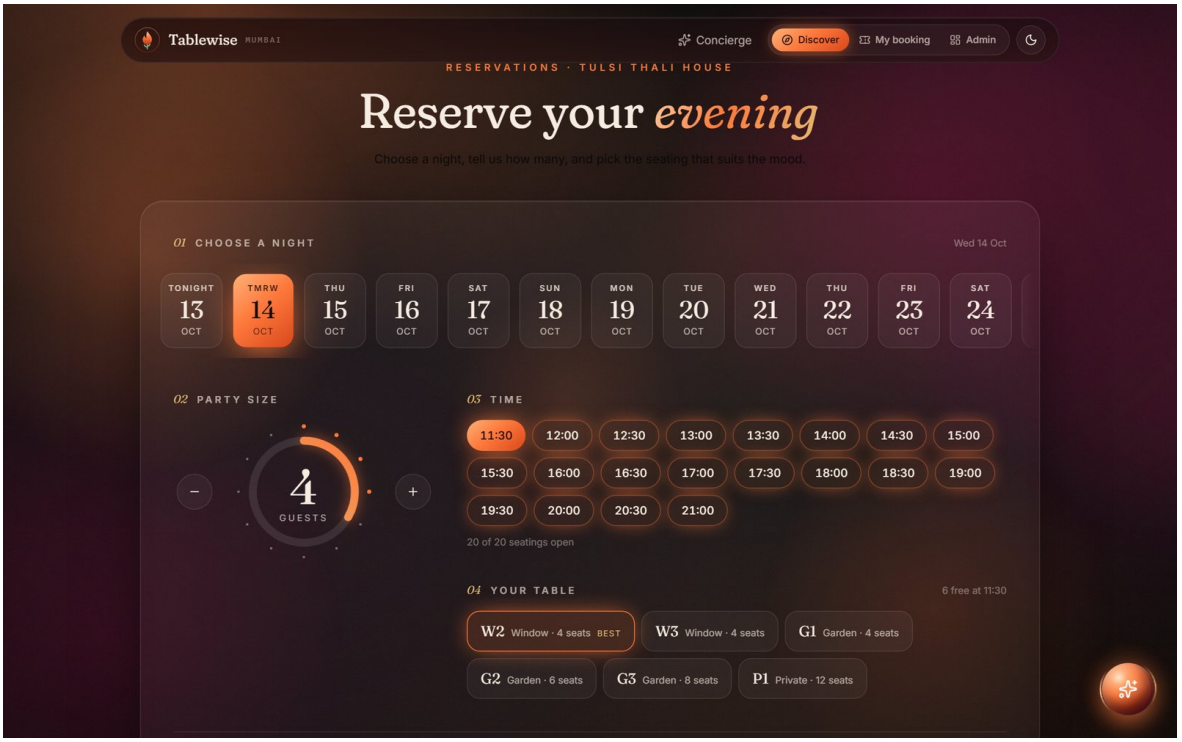


Figure 4.5: Booking panel with the preferences step

4.4.Confirmation Ticket and Booking Lookup

A successful booking shows a ticket with the booking code, restaurant, time, party, table and the guest's preferences. The same details are available at `/bookings/<code>`, where a confirmed booking can also be cancelled.

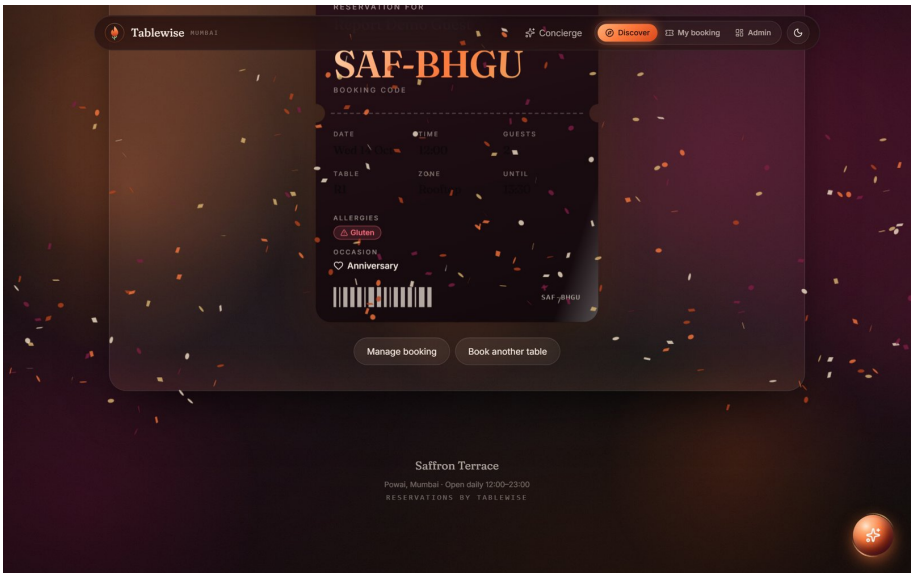


Figure 4.6: Confirmation ticket

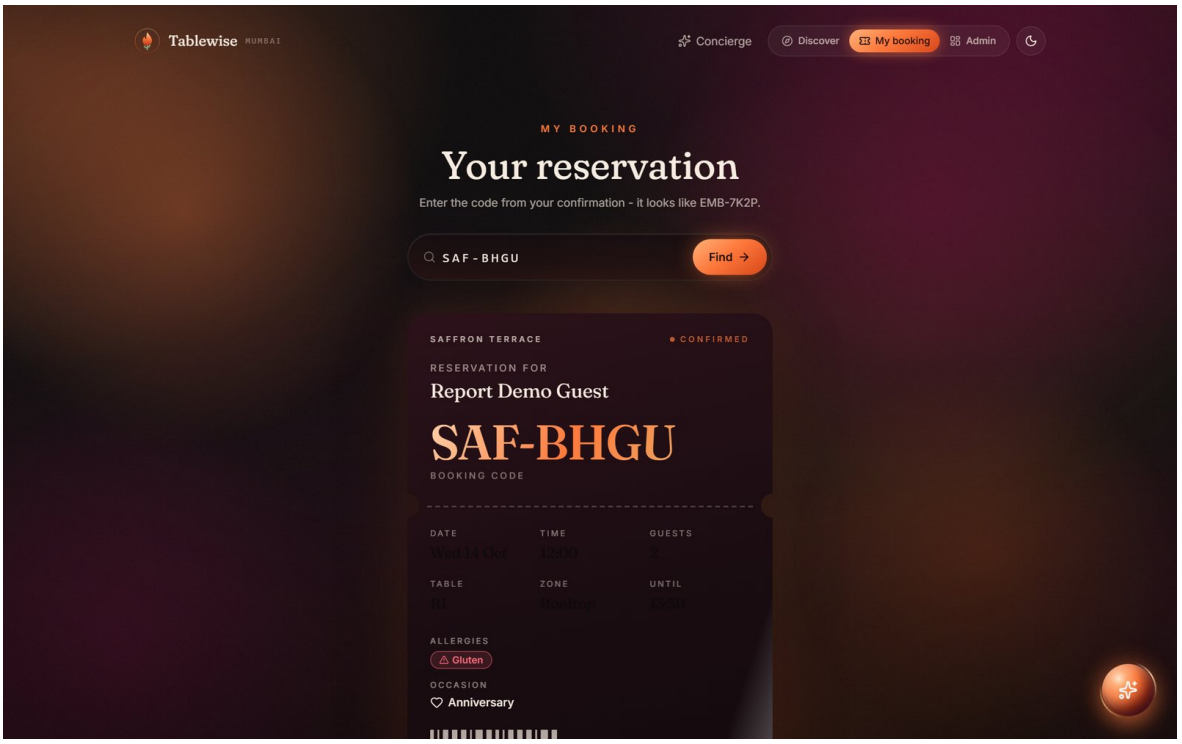


Figure 4.7: Booking lookup page

4.5.AI Concierge

The concierge drawer is available on every page and, on a restaurant page, is scoped to that restaurant. Answers stream token by token. A trace strip shows which graph node is running, tool calls appear as cards, and the documents behind an answer appear as source chips. Discovery requests return restaurant cards with bookable slots.

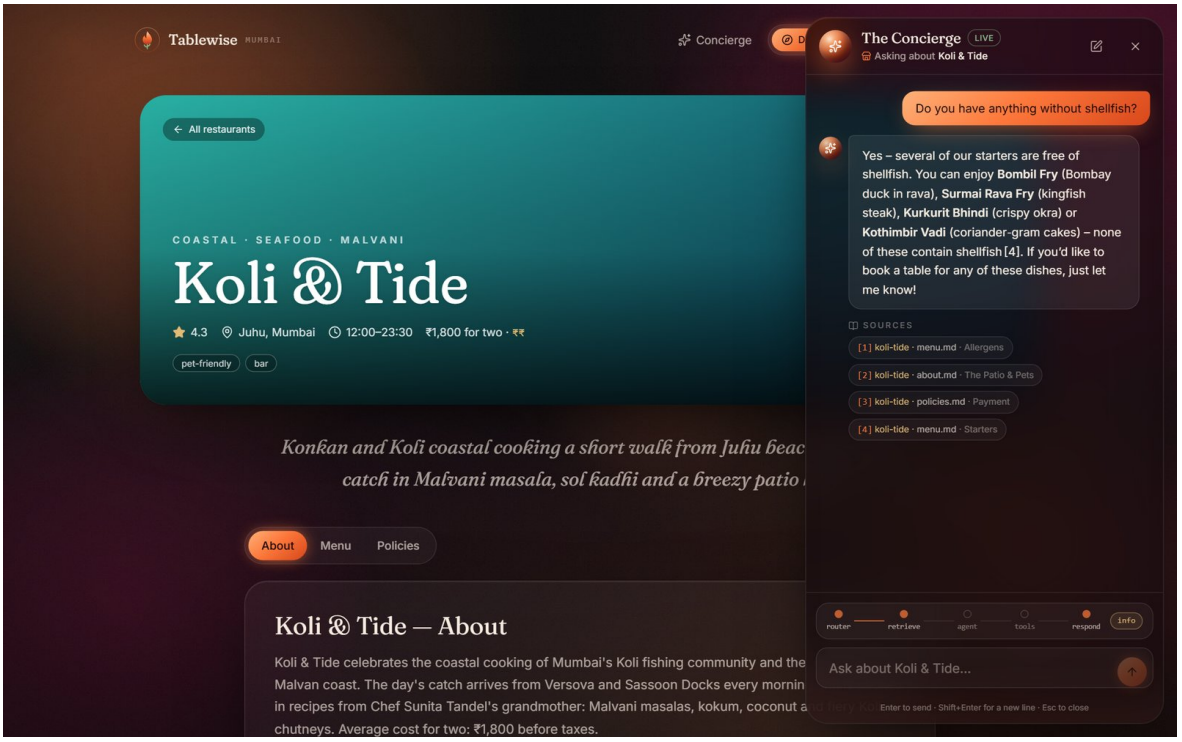


Figure 4.8: Concierge answering from restaurant documents

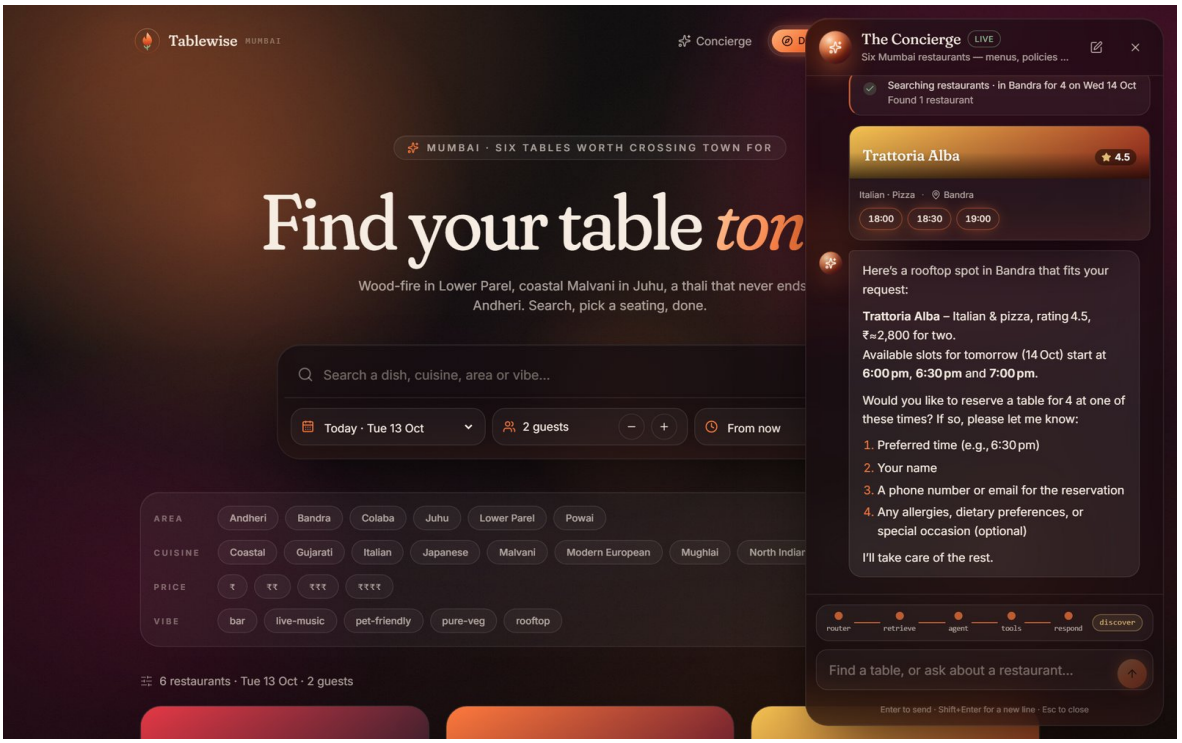


Figure 4.9: Concierge discovery results

4.6.Admin Timeline and Booking Drawer

The admin view shows one restaurant's night as a timeline with one row per table. Booking chips are coloured by status and marked with a warning icon for allergies and a cake icon for birthdays and anniversaries. Clicking a chip opens a drawer with the full booking, including allergies and dietary needs, and buttons to mark it seated, completed, no-show or to cancel it.

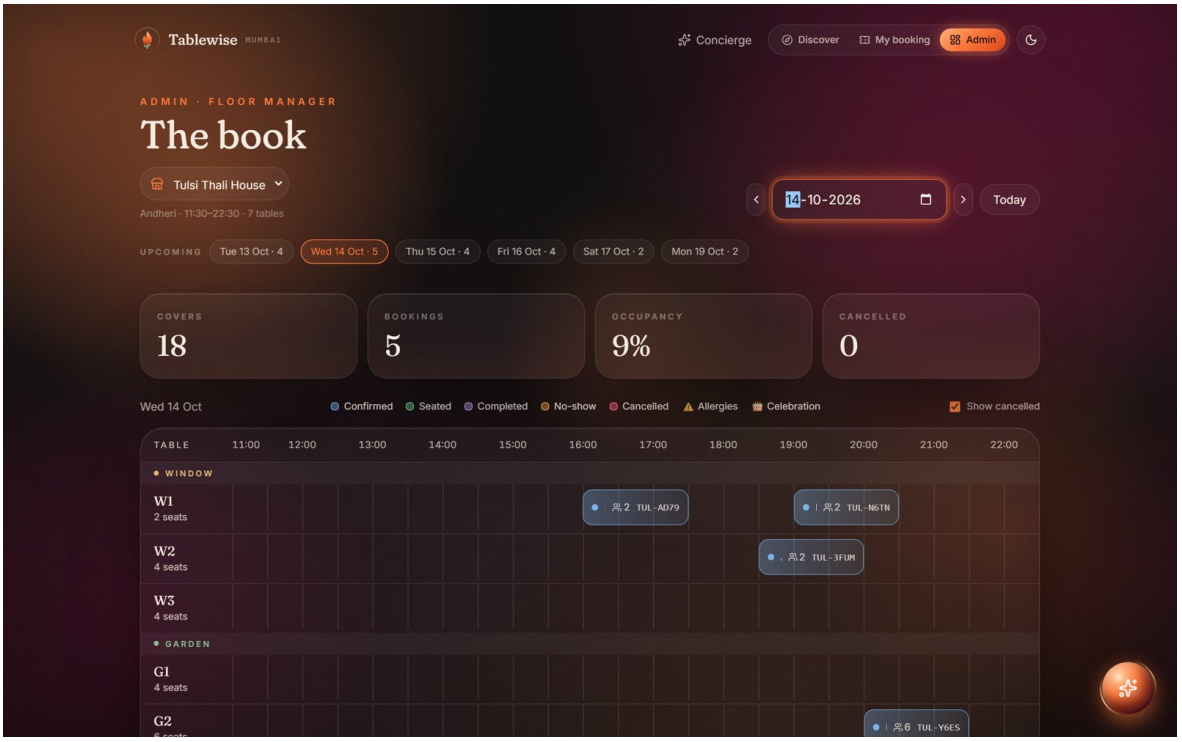


Figure 4.10: Admin timeline

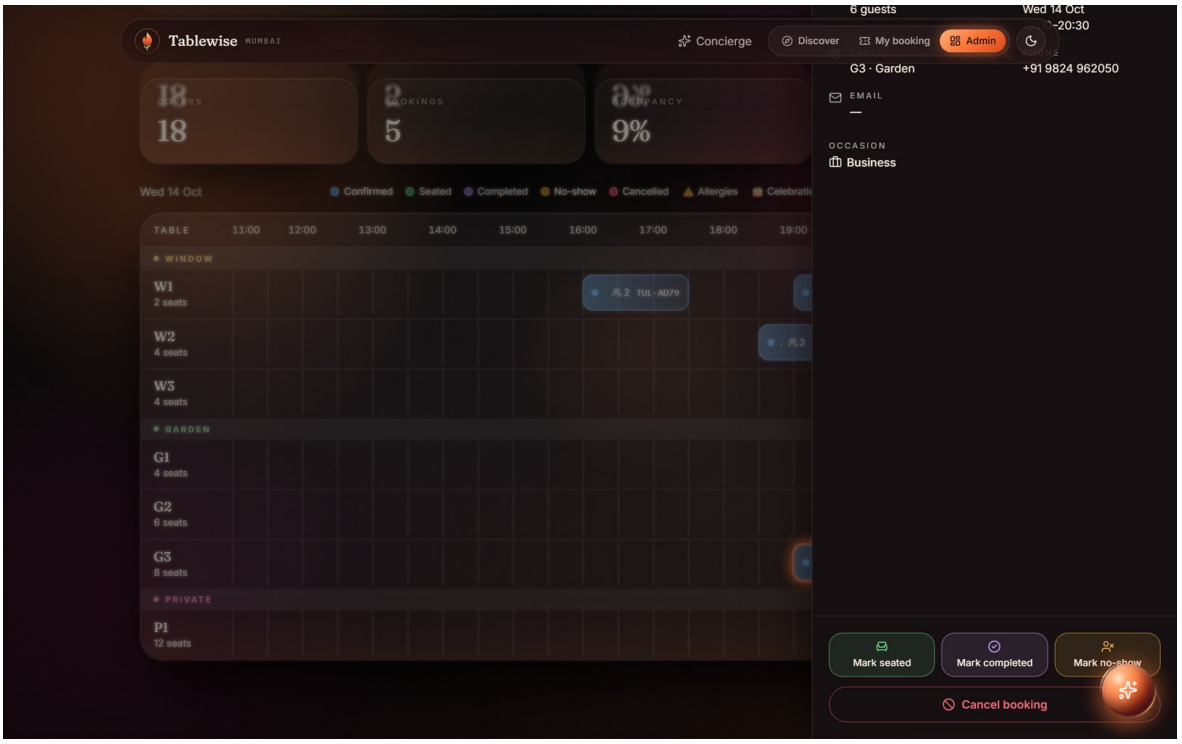


Figure 4.11: Booking details drawer

4.7.Booking Entry Page for Map Listings

`/book/<slug>` is a compact booking page intended as the reservation link for a restaurant's map listing. It carries the same JSON-LD and booking panel without the rest of the restaurant page. No integration with Google's Reserve programme exists; it is a standalone entry point.



Figure 4.12: Booking entry page

4.8.Instrumentation

The backend exposes HTTP request metrics through prometheus-fastapi-instrumentator and six custom metrics: bookings created (by restaurant and source), bookings cancelled, chat requests by intent, chat latency, RAG retrieval time and LLM errors. They are incremented inside the booking service, the chat runner and the retrieval function, so the web, chat and MCP paths are all counted.

5. DevOps Implementation

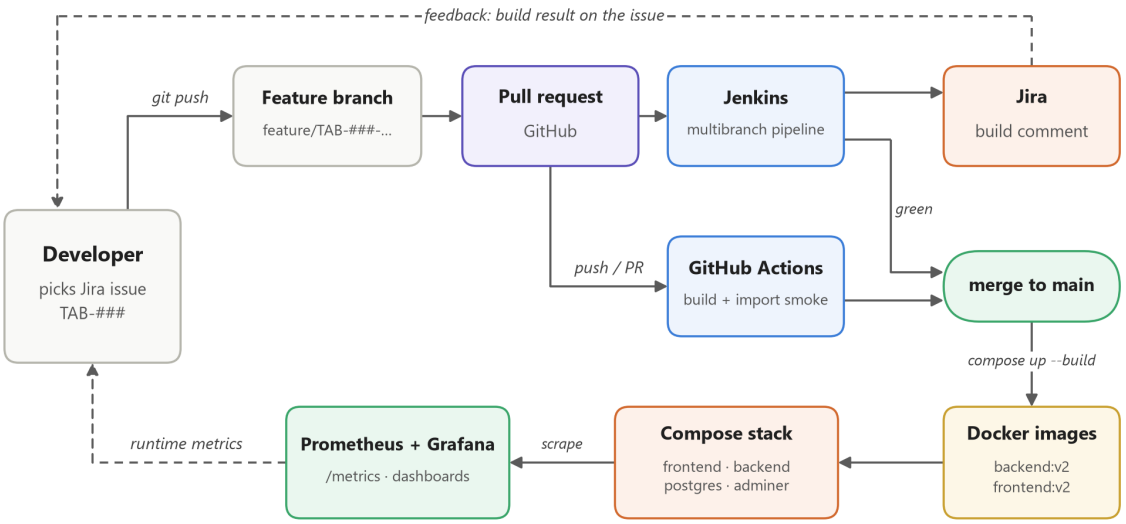


Figure 5.1: Delivery flow and feedback loop

5.1.Version Control with Git and GitHub (Experiment 1)

The code lives in the GitHub repository `aryabailur/tablewise-`. Work happens on short-lived branches named after a Jira issue (`feature/TAB-…`, `fix/TAB-…`, `chore/TAB-…`) and reaches main only through pull requests. The history contains 24 commits and 3 merge commits; 23 of the 24 commit subjects start with a Jira key. `CODEOWNERS` assigns the backend, frontend and DevOps paths to their owners, a pull request template asks for the Jira key, summary, test steps and screenshots, and main is protected so that a pull request needs 1 approving review before it can be merged.

PR	Title	Branch	Lines	State
#1	TAB-31: Booking API validation, overlap tests and off-topic guardrail	`feature/TAB-1-booking-guardrails`	+173 / −30	Open
#2	TAB-39: Admin view shows new bookings on their night	`fix/TAB-39-admin-new-bookings`	+98 / −13	Open
#3	TAB-33: Booking panel skeletons, mobile polish and frontend lint	`feature/TAB-4-booking-panel-polish`	+1837 / −202	Open
#4	TAB-5: Add DevOps teammate to CODEOWNERS and README	`chore/TAB-5-add-devops-teammate`	+18 / −6	Open
#5	TAB-40: Tablewise v2 - multi-restaurant discovery, per-restaurant RAG, Grafana	`feature/TAB-40-multi-restaurant`	+6897 / −1328	Open
#6	TAB-50: Admin booking details drawer, allergies and status tracking	`feature/TAB-50-admin-booking-details`	+7984 / −1359	Open

Table 5.1: Pull requests

Pull requests #5 and #6 are stacked on #1–#3, so they also contain those changes; this is why their line counts are large. None has been merged yet, because each needs an approving review from a teammate's own account and ahsiri14 has not yet accepted the repository invitation.

5.2.Containerisation with Docker (Experiment 2)

- **Backend image** (`backend/Dockerfile`): based on `python:3.11-slim`, installs pinned dependencies from `requirements.txt`, copies the application and knowledge documents, declares volumes for the Chroma index and embedding-model cache, and defines a health check that calls `/api/health`. The entrypoint waits for PostgreSQL, migrates and seeds the database on first start, then starts Uvicorn.
- **Frontend image** (`frontend/Dockerfile`): a multi-stage build that compiles the Vite application in `node:20-alpine` and serves the static files from `nginx:1.27-alpine`, with a health check. The Nginx configuration falls back to `index.html` for client-side routes and proxies `/api` to the backend with buffering disabled so chat responses stream.

5.3.Multi-Container Orchestration with Docker Compose (Experiment 3)

Service	Image	Host port	Role
postgres	postgres:16.4	5432 (internal)	Database, health check, named volume
backend	tablewise-backend (built)	8000	API, agent, metrics; waits for healthy Postgres
frontend	tablewise-frontend (built)	3000	Nginx serving the SPA and proxying /api
adminer	adminer:4.8.1	8081	Database browser
prometheus	prom/prometheus:v2.54.1	9090	Scrapes backend /metrics every 5 s
grafana	grafana/grafana:11.2.0	3001	Provisioned data sources and dashboard

Table 5.2: Docker Compose services

All third-party images are pinned to a version. Data that must survive a rebuild (PostgreSQL data, the Chroma index, the embedding-model cache and Grafana state) is kept in named volumes. The frontend host port can be changed with `FRONTEND_PORT` when port 3000 is already taken.

5.4.Continuous Integration with Jenkins and GitHub Actions (Experiment 4)

Jenkins runs in its own container (`jenkins/Dockerfile`), built from `jenkins/jenkins:lts-jdk17` with Python 3, Node.js 20 and the required plugins preinstalled. It is configured entirely as code with the Configuration as Code plugin: the admin user, the GitHub and Jira credentials (injected from environment variables at start-up, never written to the repository) and a multibranch pipeline job created with Job DSL. The job discovers every branch and every pull request of the repository and rescans every five minutes.

Stage	What it does	Duration on main (s)
Checkout	Print commit SHA and message	0
Backend: Install	Virtualenv and pinned dependencies	193

Stage	What it does	Duration on main (s)
Backend: Lint	Ruff 0.17.0	0
Backend: Tests	pytest with JUnit XML report	5
RAG: Ingest Smoke Test	Seed and ingest into a temporary Chroma directory, assert chunks > 0	50
Frontend: Install	npm ci	—
Frontend: Lint	ESLint	—
Frontend: Build	TypeScript check and Vite build	42
Package	Tarball of backend and frontend build	0

Table 5.3: Jenkins pipeline stages

After every build the pipeline publishes the JUnit report, archives the tarball, and posts a comment on each Jira issue whose key appears in the branch name, pull request title or latest commit, stating whether the build passed and linking to it. The Jira step never fails the build. Durations above are from the last build of main; the frontend lint stage exists only on the branches that add it, so main shows a dash.

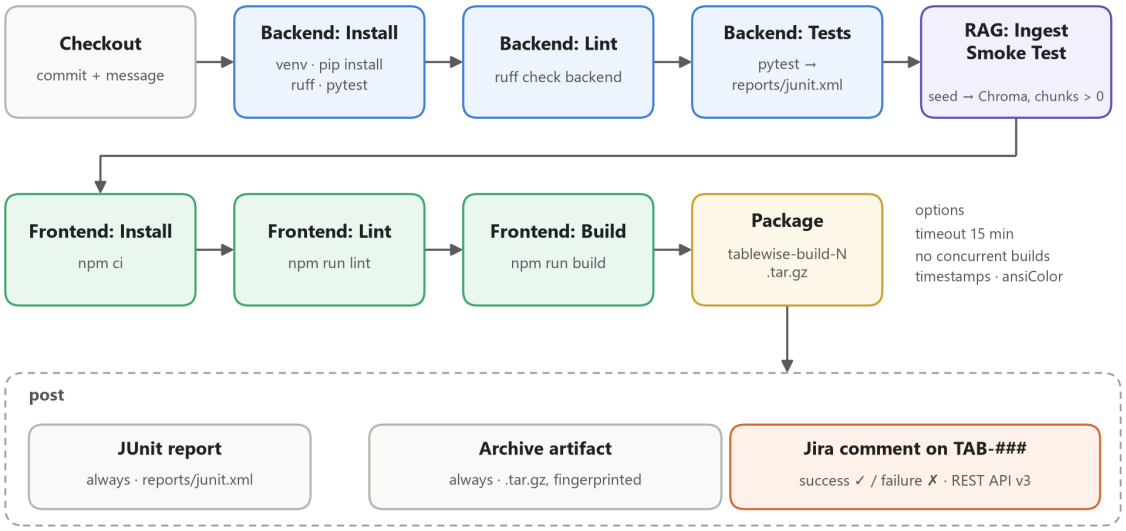


Figure 5.2: Jenkins pipeline

Jenkins has recorded 13 builds across 13 branch and pull request jobs (13 success).

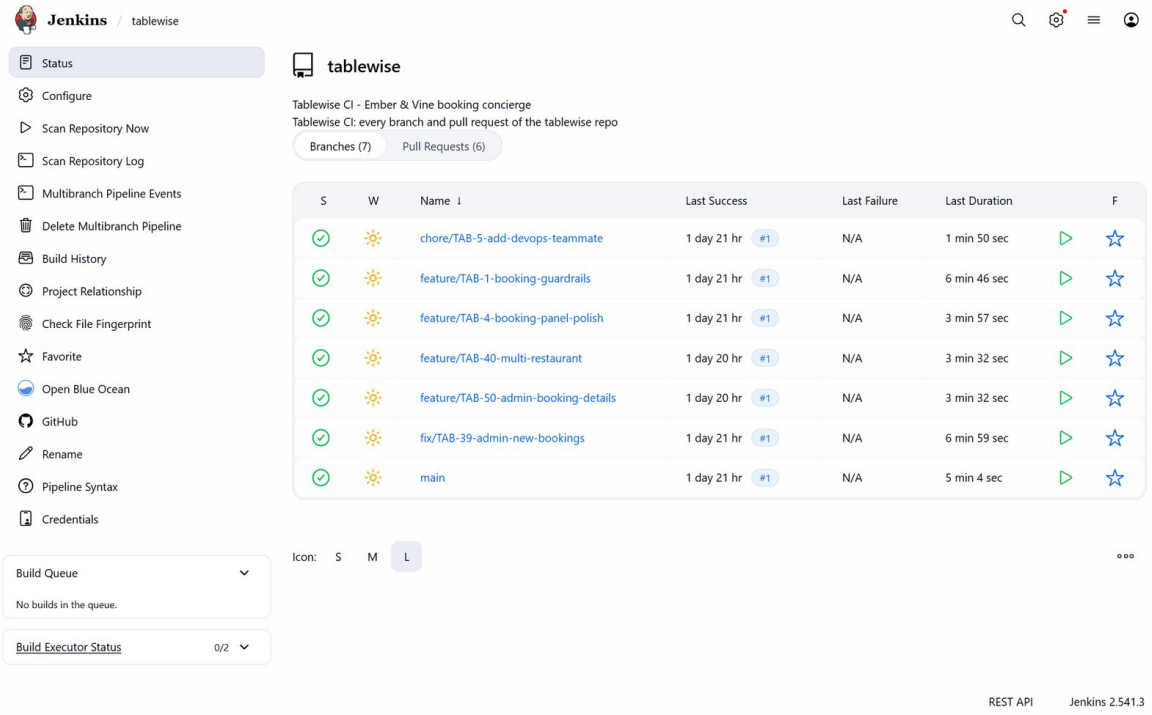


Figure 5.3: Jenkins multibranch job: branches and pull requests

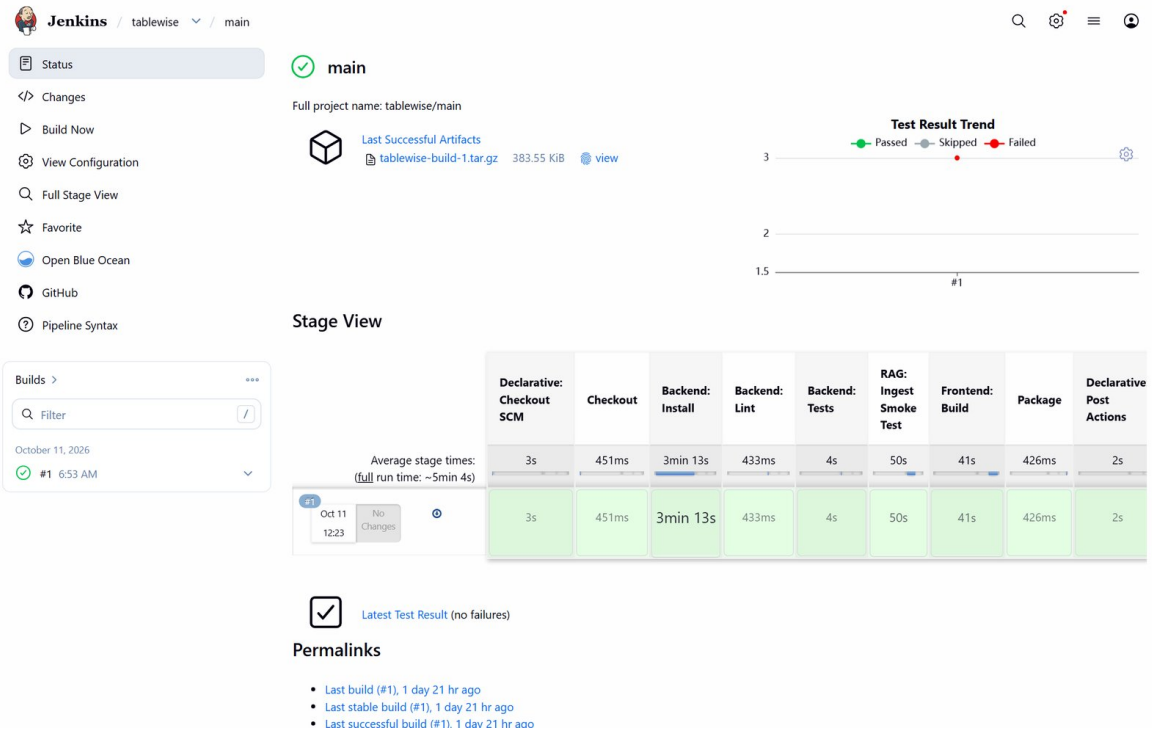


Figure 5.4: Jenkins stage view for main

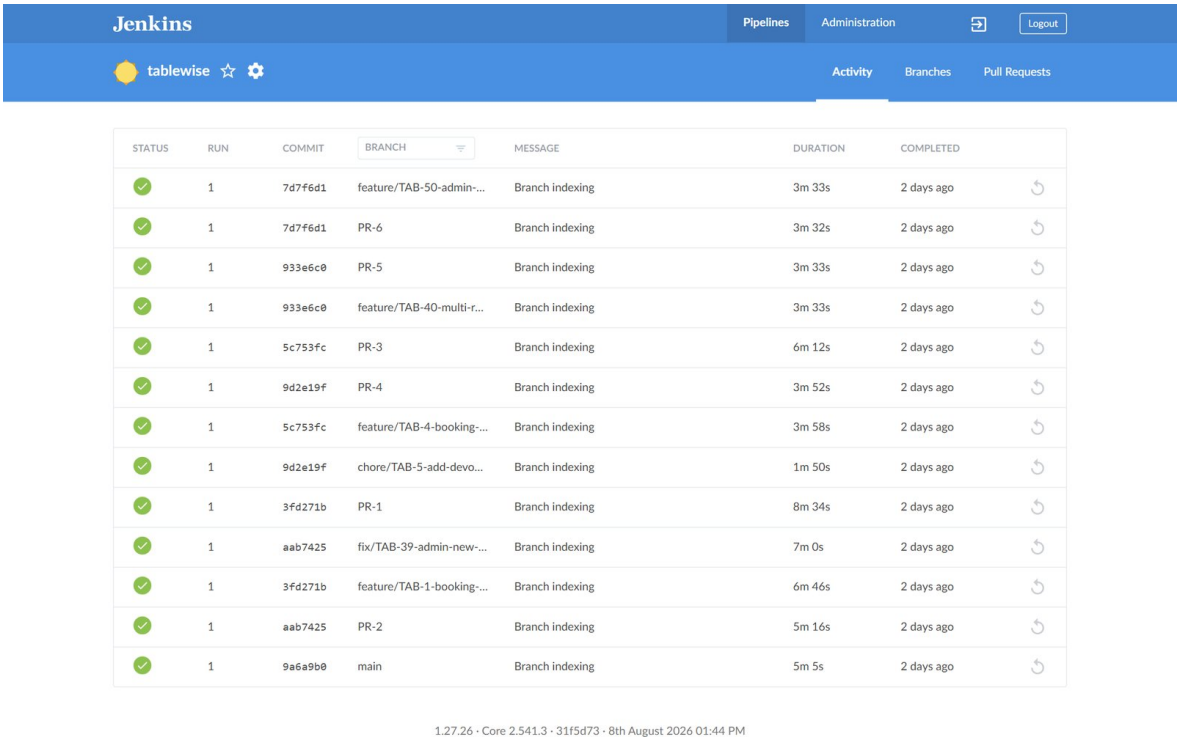


Figure 5.5: Blue Ocean activity view

GitHub Actions (`.github/workflows/ci.yml`) runs on every push and pull request: one job installs and builds the frontend, the other installs the backend with uv and imports the application and MCP server as a smoke test. It ran 7 times on the original repository `aryabailur/tablewise`, all successfully. The workflow is active on the current repository but has not produced runs there yet.

5.5.Automated Deployment (Experiment 5)

Deployment is a single `docker compose up -d --build`. Compose starts PostgreSQL first and waits for its health check; the backend entrypoint then creates or migrates the schema in place (adding new columns with defaults rather than dropping data), seeds the restaurants and demonstration bookings on an empty database, and ingests the knowledge documents. The frontend and monitoring services start once the backend is up. Each Jenkins build also packages a versioned tarball. Deploying automatically from Jenkins when main changes, with a smoke test and rollback, is not implemented and is listed as future work.

5.6.Monitoring with Prometheus and Grafana (Experiment 7)

5.6.1.Prometheus metrics

Prometheus scrapes `backend:8000/metrics` every five seconds. Besides the instrumentator's request count and latency histograms, the backend exports the custom metrics in Table 5.4.

Metric	Type	Labels
`tablewise_bookings_created_total`	Counter	restaurant, source
`tablewise_bookings_cancelled_total`	Counter	restaurant

Metric	Type	Labels
`tablewise_chat_requests_total`	Counter	intent
`tablewise_chat_latency_seconds`	Histogram	—
`tablewise_rag_retrieval_seconds`	Histogram	—
`tablewise_llm_errors_total`	Counter	—

Table 5.4: Custom application metrics

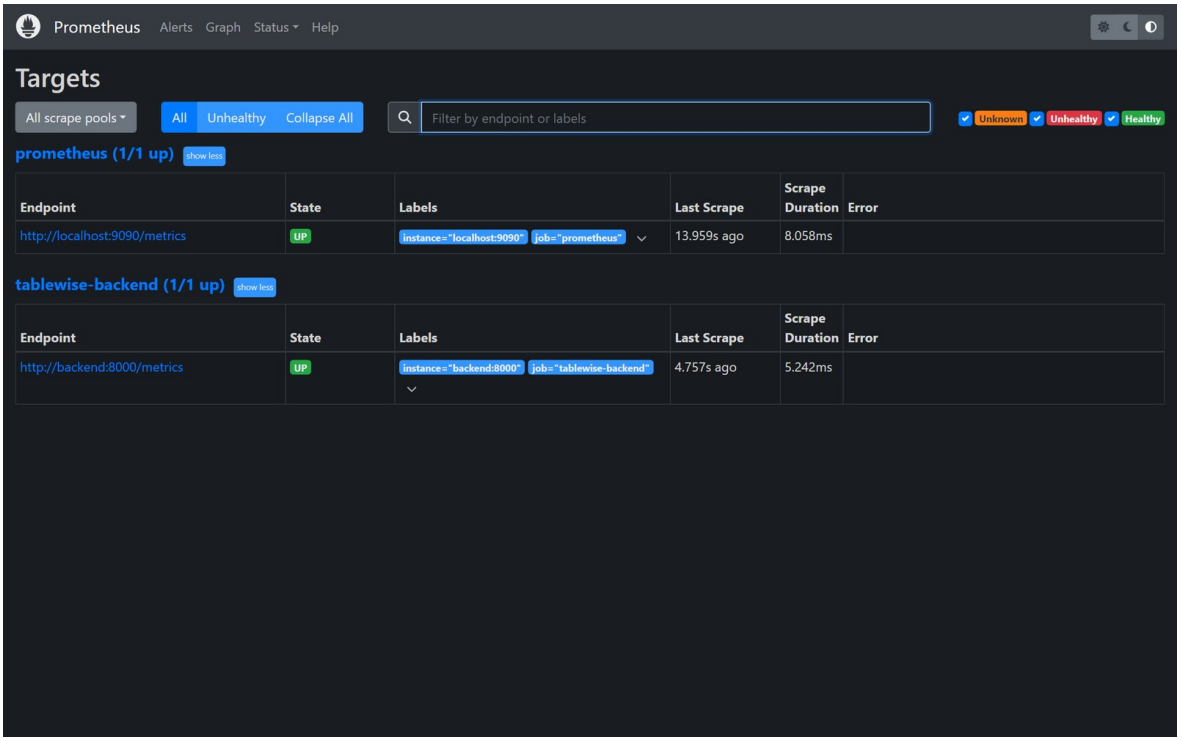


Figure 5.6: Prometheus scrape target

5.6.2.Grafana dashboard

Grafana is provisioned from files in `grafana/provisioning/`: a Prometheus data source, a PostgreSQL data source and the **Tablewise Overview** dashboard. Business panels (bookings today, by restaurant, by source and over the next seven days) query PostgreSQL directly; operational panels (requests per second, p95 latency, chat requests by intent, chat latency, RAG retrieval p95 and LLM errors) query Prometheus. No alert rules are configured yet; a chat-latency alert is open as TAB-49.



Figure 5.7: Grafana Tablewise Overview dashboard

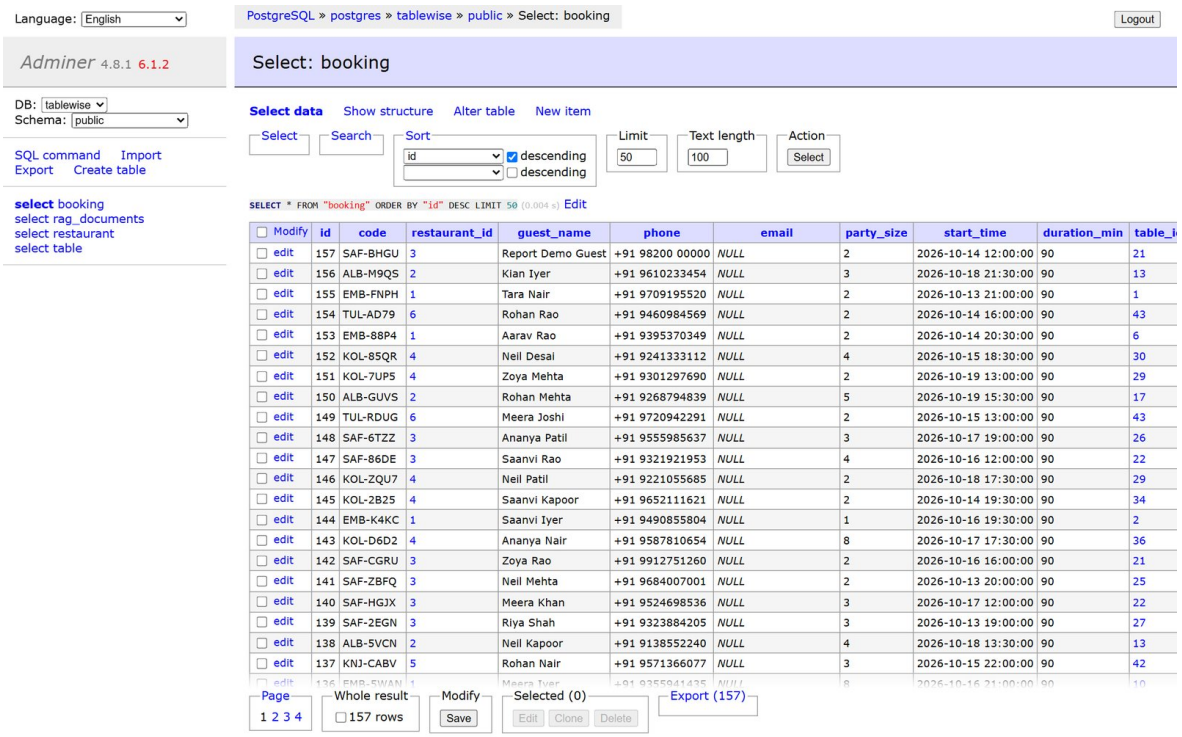


Figure 5.8: Bookings in PostgreSQL viewed through Adminer

6. Testing

6.1.Automated Tests

The backend has 42 passing test cases (24 test functions, several of them parametrised) in 4 files, run with pytest. They need no network and no LLM: booking tests use a temporary SQLite database, API tests use FastAPI's test client without running the start-up hooks, and agent tests force the offline router. Line coverage of the `app` package is 71%.

File	Test functions	What is covered
`test_api_validation.py`	3	HTTP 422 for malformed bookings, codes, dates, slugs, sources and chat input
`test_guardrail.py`	3	Off-topic short-circuit, intent heuristics, restaurant name resolution
`test_smoke.py`	5	Availability and smallest-fit table, create and cancel, overlapping bookings, restaurant isolation and search, heading chunker
`test_sprint2.py`	13	Allergy, dietary, occasion and status validation; status transitions; in-place migration of an old database; allergen cross-check

Table 6.1: Backend test files

Module	Coverage
`app/allergens.py`	100%
`app/booking_service.py`	82%
`app/db.py`	96%
`app/graph.py`	60%
`app/main.py`	84%
`app/metrics.py`	100%
`app/rag.py`	47%
`app/seed.py`	67%
Total	71%

Table 6.2: Line coverage by module

Static checks run alongside the tests: Ruff 0.17.0 on the backend (passing) and ESLint on the frontend (no errors), plus the TypeScript compiler as part of the frontend build.

6.2.End-to-End Checks

Each feature was also checked against the running stack. A booking made through the UI with nut and dairy allergies, a Jain dietary preference and a birthday occasion was stored with all four fields and appeared in the admin data; changing its status to seated through the API succeeded. A booking made through the concierge asked about allergies, named the nut-containing dishes on the restaurant's menu, and created the booking with the allergy and occasion recorded.

6.3.Load Simulation

`scripts/load_demo.py` makes random bookings across all restaurants and sends concierge chat requests so that the dashboards have data. In the run used for this report, 30 of 30 bookings were created and 20 of 20 chats completed with no error events.

Metric	Value
Bookings created	31
Bookings by source	chat: 5, google: 5, mcp: 5, phone: 5, web: 11
Chat requests by intent	booking: 1, discover: 10, info: 11
Chat latency p50 / p95	11.6 s / 55.9 s
RAG retrieval p95	0.47 s
LLM errors	0

Table 6.3: Metrics after the load simulation (Prometheus)

Latency percentiles are Prometheus histogram estimates and so are bounded by bucket edges. Chat latency is dominated by the hosted LLM: a single message can involve routing, retrieval, several tool calls and a streamed answer. Counters cover the period since the backend container last started.

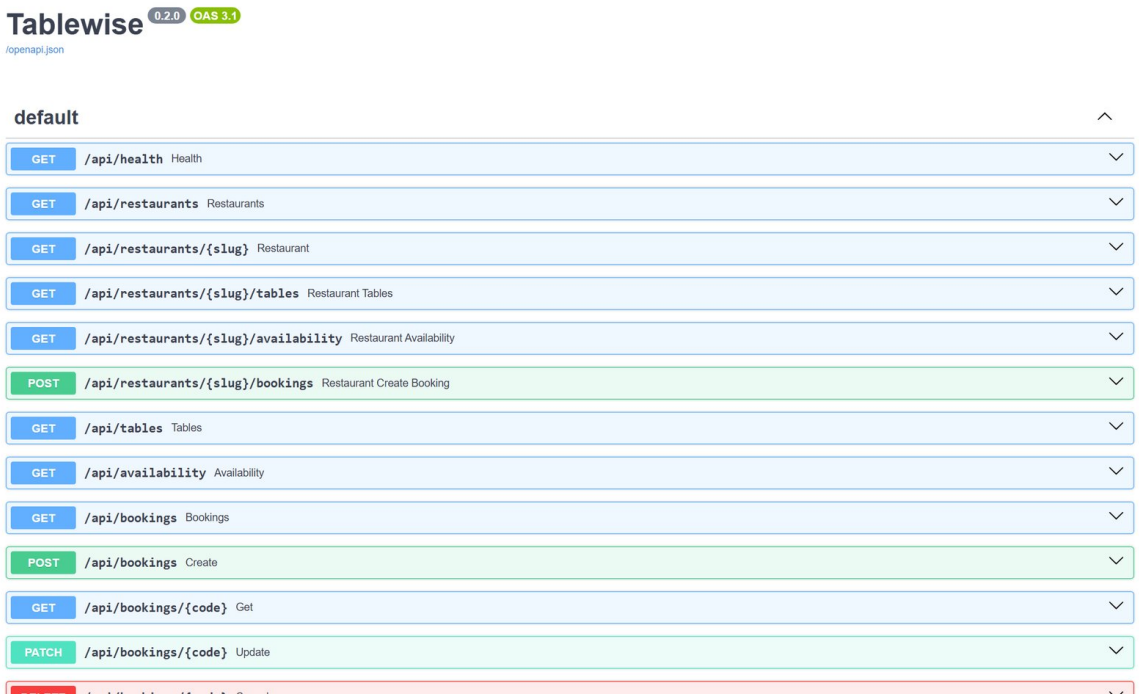


Figure 6.1: Interactive API documentation (OpenAPI)

7. Results and Discussion

7.1. Outcomes against Objectives

Objective	Outcome
Discover and book	Discover page, restaurant pages and booking flow across 6 restaurants; 157 bookings in the database at the time of writing
Dietary details	Allergies, dietary preference, occasion and requests stored and shown; 43 bookings carry allergies
AI concierge	Intent routing, per-restaurant RAG with citations, discovery and booking tools, guardrail and allergen cross-check
MCP	7 tools over stdio reusing the booking service
Host workflow	Admin timeline, details drawer and status actions (in review as PR #6)
Agile planning	54 Jira issues in 2 sprints; Sprint 1 closed with 20/20 done
CI	13 Jenkins builds, all passing; GitHub Actions workflow
Containers	6-service Compose stack started with one command
Monitoring	Prometheus metrics and a provisioned Grafana dashboard with live data

Table 7.1: Objectives and outcomes

7.2. The Feedback Loop in Practice

Sprint 1 testing of the admin view found that clicking a booking did nothing and that table names and codes were hard to read. Both were raised as bugs (TAB-50 and TAB-51) with steps to reproduce, planned into Sprint 2, implemented on a branch named after the bug, built by Jenkins, and linked back to the Jira issues by the pull request comment and the build comment. Earlier, a host-facing problem — new bookings for other nights not appearing in the admin view — went through the same loop as TAB-39.

7.3. Challenges and Resolutions

Challenge	Resolution
sqlmodel 0.0.48 rejected naive datetimes, breaking every booking	Columns declared with an explicit naive SQLAlchemy DateTime (TAB-10)
MCP SDK 2.x renamed FastMCP	Import that works with both 1.x and 2.x
Ruff 0.17 changed its default rule set	Ruff pinned in CI and rules declared in pyproject.toml
Adding allergy and status fields to a database that already held data	In-place migration that adds missing columns with defaults, with a test on an old-schema database
Host disk filled up and corrupted the Jenkins home volume	Volume recreated; configuration as code restored the whole Jenkins setup on start
Branch protection unavailable on a private repository on the free plan	Repository moved to a public repository, where protection with one required approval is enabled

Table 7.2: Challenges and how they were resolved

7.4.Limitations

- The admin view and booking changes are not authenticated; anyone with a booking code can cancel it (TAB-29).
- All commits were authored from one account, so Git history does not show the contributions of each member.
- None of the six pull requests has been merged yet; main still runs the first single-restaurant version.
- There is no automatic deployment from Jenkins and no alerting.
- Story points were recorded on only a few issues, so velocity cannot be reported in points.
- TAB-24 (interactive floor plan) is marked Done in Jira, but no floor plan exists in the code; tables are chosen from a list.

8. Conclusion and Future Scope

8.1.Conclusion

Tablewise meets its functional goal: a guest can find a restaurant, book a table with their allergies and occasion recorded, and do the same in conversation with an AI concierge that answers from each restaurant's own documents; a host can see the night's bookings and their details. The same booking logic serves the web interface, the concierge and external AI clients through MCP.

On the process side, every lab experiment is exercised on real work: Jira issues drove the branches, commits and pull requests; every branch and pull request was built by Jenkins; the system runs as a Docker Compose stack; and Prometheus and Grafana show how it behaves. The clearest lesson was the value of the feedback loop — bugs found in testing returned to the backlog with enough detail to be fixed and traced to a build.

8.2.Future Scope

- Authentication for hosts and guests, with the admin view restricted to restaurant staff.
- Automatic deployment from Jenkins on merge to main, with a smoke test and rollback.
- Grafana alert rules (for example on chat latency) and availability monitoring with Nagios.
- Booking confirmation and reminder notifications by email or SMS (TAB-36).
- An interactive floor plan for choosing a table.
- Integration with map listings through Reserve with Google.
- Each team member committing and reviewing from their own account, so history reflects individual work.

References

- [1] FastAPI documentation, https://fastapi.tiangolo.com
- [2] LangGraph documentation, https://langchain-ai.github.io/langgraph
- [3] Chroma documentation, https://docs.trychroma.com
- [4] Model Context Protocol specification, https://modelcontextprotocol.io
- [5] Jenkins Pipeline and Configuration as Code documentation, https://www.jenkins.io/doc
- [6] Docker and Docker Compose documentation, https://docs.docker.com
- [7] Prometheus documentation, https://prometheus.io/docs
- [8] Grafana provisioning documentation, https://grafana.com/docs/grafana/latest/administration/provisioning
- [9] Jira Software Cloud REST API, https://developer.atlassian.com/cloud/jira/software
- [10] K. Schwaber and J. Sutherland, The Scrum Guide, 2020.

Appendix A: Running the Project

- Clone `https://github.com/aryabailur/tablewise-` and copy `backend/.env.example` to `backend/.env`, adding a Groq API key (without a key the concierge runs in a limited offline mode).
- Start everything: `docker compose up -d --build` (set `FRONTEND_PORT` if port 3000 is in use).
- Generate demonstration traffic: `python scripts/load_demo.py`.
- Open the app on port 3000, the API documentation at `http://localhost:8000/docs`, Grafana at `http://localhost:3001` (admin / admin), Prometheus at `http://localhost:9090` and Adminer at `http://localhost:8081`.
- Start Jenkins with `JIRA_API_TOKEN=… ./jenkins/run.sh` and open `http://localhost:8080` (admin / admin).
- Run the backend tests: `cd backend && uv run --with pytest python -m pytest -q`.